🏢 Smart Property Rental & Maintenance Platform

Advanced Real-World Python Backend Project

Project objective: Build a backend system for property owners and property management companies to manage apartments, tenants, rental agreements, rent payments, maintenance complaints, technicians, and financial reports.

Students can use FastAPI, Django REST Framework, or Flask with:

- Python + OOP
- PostgreSQL
- MongoDB
- REST APIs
- Authentication
- Testing
- GitHub

1. Understand the Real-World Problem

Scenario

A property management company manages:

- 5 apartment buildings
- 200 apartments
- 150 tenants
- 20 maintenance technicians

Currently, the company uses spreadsheets and messaging applications to manage everything.

Existing problems

1. Tenants forget rent payment dates.
2. Owners cannot easily track monthly income.
3. Maintenance complaints are lost in messages.
4. Technicians are assigned manually.

5. Duplicate complaints are created.
6. Managers cannot track unresolved issues.
7. No proper history exists for rent and maintenance activities.

Your solution

Build a centralized backend API that manages the complete rental and maintenance lifecycle.

2. Complete System Workflow

Property Management Workflow

1. Register Property
   Admin adds buildings and apartments.
2. Register Tenant
   Staff creates tenant records using mock data.
3. Create Rental Agreement
   Tenant is assigned an apartment with rent and agreement dates.
4. Generate Rent Due Record
   System creates monthly rent obligations.
5. Track Rent Payment
   Payment is recorded and the rent status is updated.
6. Submit Maintenance Complaint
   Tenant reports a problem in their apartment.
7. Assign Technician
   Manager assigns an appropriate technician.
8. Track Repair
   Complaint progresses from open to resolved.
9. Generate Reports
   Manager views rental income, overdue rent, and maintenance performance.

3. User Roles

|
Role

|

Responsibilities

|
| --- | --- |
|

Admin

|

Manage properties, users, and configuration

|
|

Property Manager

|

Manage tenants, agreements, rent, and complaints

|
|

Tenant

|

View agreement, rent, and submit complaints

|
|

Technician

|

View assigned jobs and update repair status

|
|

Owner

|

View property income and performance reports

|

Beginner hint

Implement Admin, Property Manager, Tenant, and Technician first. Add Owner dashboards as an advanced feature.

4. Functional Modules

Module 1: Property and Apartment Management

Objective

Manage buildings, apartments, and their availability.

Property fields

- Property ID
- Property name
- Address reference
- Owner reference
- Number of apartments
- Property status

Apartment fields

- Apartment ID
- Property ID
- Apartment number
- Floor number
- Apartment type
- Monthly rent
- Status

Apartment statuses

AVAILABLE
OCCUPIED
UNDER_MAINTENANCE
INACTIVE

Required APIs

|
Method

|

Endpoint

|

Purpose

|
| --- | --- | --- |
|

POST

|

`/properties`

|

Create property

|
|

GET

|

`/properties`

|

List properties

|
|

POST

|

`/properties/{id}/apartments`

|

Add apartment

|
|

GET

|

`/properties/{id}/apartments`

|

List apartments

|
|

PATCH

|

`/apartments/{id}`

|

Update apartment

|
|

GET

|

`/apartments?status=AVAILABLE`

|

Filter apartments

|

Business rules

1. Apartment number must be unique within a property.
2. An occupied apartment cannot be assigned to another active tenant.
3. An apartment under maintenance may have restricted assignment.
4. Monthly rent must be greater than zero.

5. Module 2: Tenant Management

Objective

Register and manage tenant information.

Tenant fields

tenant_id
full_name
email
phone_reference
identity_reference
registration_date
status

Use mock contact and identity references rather than real personal documents.

Required APIs

http

POST /tenants
GET /tenants
GET /tenants/{tenant_id}
PATCH /tenants/{tenant_id}
PATCH /tenants/{tenant_id}/deactivate

Business rules

- Email must be valid.
- A deactivated tenant cannot create a new rental agreement.
- A tenant may have multiple historical agreements.
- A tenant should not have overlapping active agreements unless explicitly supported.

OOP hint

Tenant

TenantService
TenantRepository

Keep validation and business rules inside the service layer.

6. Module 3: Rental Agreement Management

Objective

Connect tenants with apartments through rental agreements.

Rental agreement fields

|
Field

|

Description

|
| --- | --- |
|

Agreement ID

|

Unique agreement identifier

|
|

Tenant ID

|

Tenant reference

|
|

Apartment ID

|

Apartment reference

|
|

Start date

|

Agreement beginning

|
|

End date

|

Agreement expiry

|
|

Monthly rent

|

Agreed rent

|
|

Security deposit

|

Mock amount

|
|

Status

|

Active, expired, terminated

|

Required APIs

http

```
POST   /rental-agreements
GET    /rental-agreements
GET    /rental-agreements/{agreement_id}
PATCH  /rental-agreements/{agreement_id}/terminate
GET    /apartments/{apartment_id}/current-tenant
```

Important business rules

1. Apartment must be available before creating an active agreement.
2. An apartment cannot have overlapping active rental agreements.
3. Agreement end date must be after the start date.
4. Monthly rent must be recorded.
5. Terminating an agreement should update apartment availability according to the business policy.

Conflict detection hint

Check whether an existing active agreement overlaps the proposed agreement dates:

```
new_start < existing_end
AND
new_end > existing_start
```

This is similar to appointment conflict detection but applies to date ranges.

7. Module 4: Rent and Payment Management

Objective

Track monthly rent obligations and payments.

Example

A tenant has monthly rent of ₹18,000.

October Rent
Amount Due: ₹18,000
Due Date: 5 October
Status: PENDING

After payment:

Amount Paid: ₹18,000
Status: PAID

Suggested tables

Rent obligations

- Rent ID
- Agreement ID
- Billing month
- Due date
- Amount due
- Amount paid
- Status

Payments

- Payment ID
- Rent ID
- Amount
- Payment date
- Payment method
- Reference number
- Status

Rent statuses

PENDING
PARTIALLY_PAID
PAID
OVERDUE
WAIVED

Required APIs

http

POST /rent-obligations/generate
GET /rent-obligations
GET /tenants/{tenant_id}/rent
POST /payments
GET /payments/{payment_id}
GET /reports/overdue-rent

Business rules

1. Payment amount must be positive.
2. Payment cannot exceed the outstanding amount unless overpayment is explicitly supported.
3. Paid rent cannot be marked overdue.
4. A payment must reference a valid rent obligation.
5. Monthly rent obligations should not be duplicated.

Advanced feature

Automatic overdue detection

A scheduled task checks unpaid obligations after their due date and changes their status to `OVERDUE`.

You can initially run this through an API or command, then implement scheduled processing later.

8. Module 5: Maintenance Complaint Management

This is the primary operational module.

Real-world scenario

A tenant reports:

> "Water leakage in the kitchen."

The property manager must track the complaint and assign a technician.

Complaint fields

- Ticket ID

- Tenant ID
- Apartment ID
- Complaint category
- Description
- Priority
- Status
- Created timestamp
- Assigned technician
- Resolution timestamp

Complaint categories

PLUMBING
ELECTRICAL
CARPENTRY
CLEANING
APPLIANCE
STRUCTURAL
OTHER

Complaint priority

LOW
MEDIUM
HIGH
CRITICAL

Complaint status

OPEN
ASSIGNED
IN_PROGRESS
ON_HOLD
RESOLVED
CLOSED
CANCELLED

Required APIs

|
Method

|

Endpoint

|

Purpose

|
| --- | --- | --- |
|

POST

|

`/maintenance-tickets`

|

Create complaint

|
|

GET

|

`/maintenance-tickets`

|

List complaints

|
|

GET

|

`/maintenance-tickets/{id}`

|

View complaint

|
|

PATCH

|

`/maintenance-tickets/{id}/assign`

|

Assign technician

|
|

PATCH

|

`/maintenance-tickets/{id}/start`

|

Start repair

|
|

PATCH

|

`/maintenance-tickets/{id}/resolve`

|

Resolve complaint

|
|

PATCH

|

`/maintenance-tickets/{id}/close`

|

Close ticket

|
|

GET

|

`/tenants/{id}/maintenance-tickets`

|

Tenant complaints

|

Business rules

1. A complaint must belong to a valid apartment.
2. The tenant must be associated with that apartment.
3. Only an assigned technician can start the repair, depending on role permissions.
4. A resolved complaint must include a resolution note.
5. A closed complaint cannot be reopened without an authorized action.
6. Critical complaints should receive a shorter target resolution time.

9. Module 6: Technician Management

Objective

Assign maintenance work to suitable technicians.

Technician fields

technician_id
name
specialization
availability_status
contact_reference

Specializations

PLUMBING
ELECTRICAL

CARPENTRY
GENERAL

Required APIs

http

POST /technicians
GET /technicians
GET /technicians/{id}/assignments
PATCH /technicians/{id}/availability
POST /maintenance-tickets/{id}/assign

Technician assignment logic

The system can consider:

1. Technician specialization.
2. Availability.
3. Number of active assignments.
4. Distance, if simulated.
5. Complaint priority.

Example

Complaint: Water Leakage
Category: PLUMBING

Available Technicians:
T001 - Electrical - Available
T002 - Plumbing - Available
T003 - Plumbing - Busy

Suggested Technician: T002

Hint

Start with rule-based assignment. Students do not need machine learning.

10. Module 7: Maintenance SLA Tracking

What is an SLA?

An SLA (Service Level Agreement) defines the expected response or resolution time for a complaint.

Example SLA policy

|
Priority

|

Response Target

|

Resolution Target

|
| --- | --- | --- |
|

Critical

|

15 minutes

|

4 hours

|
|

High

|

1 hour

|

12 hours

|
|

Medium

|

4 hours

|

24 hours

|
|

Low

|

1 business day

|

3 days

|

These are sample project policies, not universal industry standards.

Advanced features

- Track time from creation to assignment.
- Track time from assignment to resolution.
- Mark SLA breaches.
- Generate technician performance reports.
- Display unresolved high-priority tickets.

OOP hint

```
SLAService
   calculate_response_time()
   calculate_resolution_time()
   check_sla_breach()
```

11. Module 8: MongoDB Activity and Audit Logs

Why MongoDB?

PostgreSQL stores structured business data. MongoDB can store flexible activity events, complaint updates, and communication records.

Events to store

- Property created
- Tenant registered
- Rental agreement created
- Rent payment recorded
- Complaint created
- Technician assigned
- Complaint status changed
- Complaint resolved
- Agreement terminated

Example document

JSON

```
{
  "event_id": "EVT-10001",
  "event_type": "MAINTENANCE_STATUS_CHANGED",
  "entity_type": "MAINTENANCE_TICKET",
  "entity_id": "TKT-2001",
  "actor_reference": "TECH-001",
  "timestamp": "2026-10-10T10:30:00Z",
  "metadata": {
    "old_status": "IN_PROGRESS",
    "new_status": "RESOLVED",
    "resolution_note": "Leakage repaired"
  }
}
```

Suggested MongoDB collections

audit_logs
maintenance_events
tenant_communications
document_metadata

Requirements

- Store event timestamps.

- Use indexes for entity IDs and timestamps.
- Preserve audit history.
- Avoid storing unnecessary personal or confidential information.

12. PostgreSQL Database Design

Core tables

users
roles
properties
apartments
tenants
technicians
rental_agreements
rent_obligations
payments
maintenance_tickets
maintenance_assignments
sla_policies

Important relationships

Property 1 ——— Many Apartments

Apartment 1 ——— Many Rental Agreements

Tenant 1 ——— Many Rental Agreements

Rental Agreement 1 ——— Many Rent Obligations

Rent Obligation 1 ——— Many Payments

Apartment 1 ——— Many Maintenance Tickets

Technician 1 ——— Many Maintenance Assignments

Database hints

- Use primary and foreign keys.
- Add unique constraints where required.
- Use transactions for payment and agreement operations.
- Add indexes to ticket status, tenant ID, and due date.

- Use database migrations.
- Keep financial values in a suitable precise numeric type rather than floating-point types.

13. Python OOP Architecture

Recommended classes

Property
Apartment
Tenant
RentalAgreement
RentObligation
Payment
MaintenanceTicket
Technician
SLA

PropertyService
TenantService
RentalService
RentService
MaintenanceService
TechnicianAssignmentService
SLAService
AuditService

Service responsibility example

MaintenanceService
├── create_ticket()
├── validate_tenant_apartment()
├── assign_technician()
├── update_ticket_status()
├── resolve_ticket()
└── close_ticket()

OOP requirements

- Use classes to represent important entities.
- Use services for business logic.
- Use repositories for database access.
- Use custom exceptions for business failures.

- Use dependency injection for database sessions and services.
- Avoid placing all logic inside `main.py`.

14. API Development Order

Follow this sequence to prevent confusion.

1. Step 1: Health Check
   Create `GET /health`.
2. Step 2: Property and Apartment APIs
   Add and retrieve properties and apartments.
3. Step 3: Tenant APIs
   Register tenants and retrieve tenant details.
4. Step 4: Rental Agreement APIs
   Assign apartments and prevent agreement conflicts.
5. Step 5: Rent APIs
   Generate rent obligations and record payments.
6. Step 6: Maintenance APIs
   Create complaints and update their status.
7. Step 7: Technician APIs
   Assign technicians and track repair progress.
8. Step 8: MongoDB Audit APIs
   View activity history.
9. Step 9: Reports and Advanced Features
   Add SLA reports, overdue rent, and workload analytics.

15. Suggested Four-Week Development Plan

1

- Understand the problem
- Create GitHub repository
- Design PostgreSQL tables
- Set up backend framework
- Build property, apartment, and tenant APIs

2

- Rental agreements

- Apartment availability validation
- Monthly rent obligations
- Payment tracking
- Overdue-rent logic

3

- Maintenance tickets
- Technician management
- Assignment workflow
- SLA tracking
- MongoDB audit events

4

- Unit and API tests
- Role-based authorization
- Reports
- Docker setup
- README and architecture diagrams
- Final demonstration

16. Advanced Features

Each student or team should implement at least two advanced features.

Choose Advanced Features

0 selected

Automatic overdue rent detection

Rule-based technician assignment

Maintenance SLA breach tracking

Duplicate complaint detection

Monthly property revenue analytics

Tenant maintenance history

Apartment occupancy analytics

Docker Compose deployment

Role-based access control

Scheduled monthly rent generation

17. Testing Requirements

Minimum: 20 meaningful test cases.

|
No.

|

Test

|
| --- | --- |
|

1

|

Create a valid property

|
|

2

|

Reject duplicate apartment number

|
|

3

|

Register a valid tenant

|
|

4

|

Reject invalid tenant email

|
|

5

|

Create a valid rental agreement

|
|

6

|

Reject overlapping rental agreement

|
|

7

|

Prevent assigning occupied apartment

|
|

8

|

Generate monthly rent obligation

|
|

9

|

Prevent duplicate rent obligation

|
|

10

|

Record valid payment

|
|

11

|

Reject payment greater than outstanding amount

|
|

12

|

Mark unpaid rent as overdue

|
|

13

|

Create maintenance ticket

|
|

14

|

Reject tenant-apartment mismatch

|
|

15

|

Assign a suitable technician

|
|

16

|

Reject invalid status transition

|
|

17

|

Resolve ticket with resolution note

|
|

18

|

Detect SLA breach

|
|

19

|

Store MongoDB audit event

|
|

20
|
Verify role-based API authorization
|

18. Final Demonstration Scenario

Students must demonstrate the following:

1. Create a property with three apartments.
2. Register two tenants.
3. Assign one apartment to a tenant.
4. Attempt to assign the same apartment to another tenant and show the rejection.
5. Create the monthly rent obligation.
6. Record a partial payment.
7. Display the outstanding rent.
8. Submit a plumbing complaint.
9. Assign a plumbing technician.
10. Update the ticket to `IN_PROGRESS`.
11. Resolve the complaint with a resolution note.
12. Demonstrate an SLA status.
13. Display MongoDB activity logs.
14. Generate an overdue-rent or maintenance report.

19. GitHub Submission Structure

```
smart-property-management/
│
├── app/
│   ├── api/
│   ├── core/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   ├── repositories/
│   └── db/
│
├── tests/
├── migrations/
```

```
├── sample_data/
├── docs/
│   ├── architecture.png
│   ├── er-diagram.png
│   └── api-examples.md
│
├── .env.example
├── requirements.txt
├── README.md
├── docker-compose.yml
└── .gitignore
```

README requirements

- Real-world problem
- Features and business rules
- Technology stack
- Database schema
- API documentation
- Setup instructions
- Environment configuration
- Test execution
- Screenshots or demo video
- Advanced features
- Known limitations
- Future improvements